

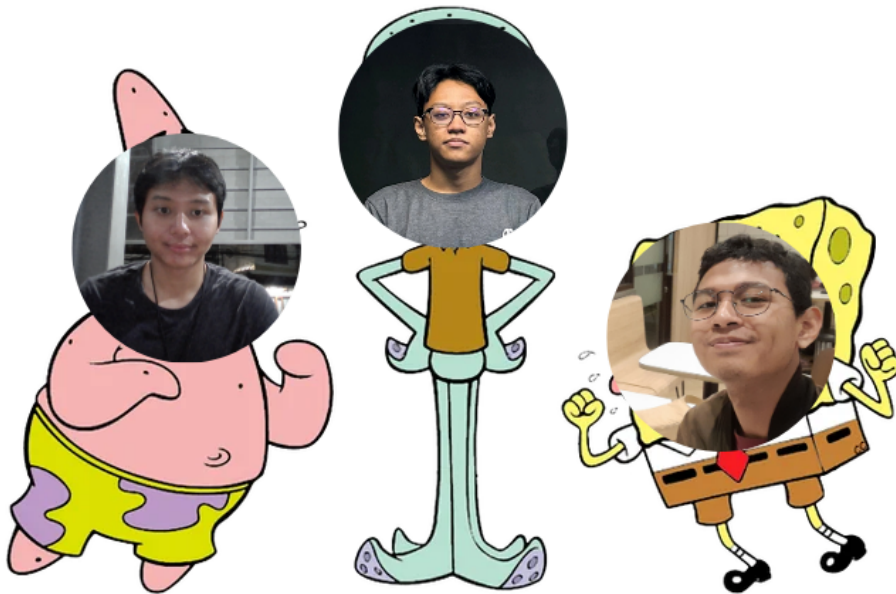
Tugas Besar 2

nama_kelompok

IF2211 Strategi Algoritma

DOMTracer : BFS-DFS Based CSS Selector Engine

Semester 2 2025/2026



Disusun oleh:

nama_kelompok

Moh. Hafizh Irham Perdana	(13524025)
Aufa Rienaldifaza Ahmad	(13524027)
Muhammad Aufar Rizqi Kusuma	(13524061)

Laboratorium Ilmu dan Rekayasa Komputasi
Program Studi Teknik Informatika
Sekolah Teknik Elektro dan Informatika
Institut Teknologi Bandung

Daftar Isi

1	Pendahuluan	3
1.1	Deskripsi Tugas	3
1.2	Spesifikasi Wajib	6
1.3	Spesifikasi Bonus	7
2	Dasar Teori	9
2.1	Document Object Model	9
2.2	HTML Parsing	9
2.3	CSS Selector	10
2.4	Traversal Graf pada Pohon DOM	10
2.4.1	Breadth-First Search	10
2.4.2	Depth-First Search	11
2.5	Pencarian Elemen dan Metrik Traversal	11
2.6	Aplikasi Web dan Pertukaran Data	11
3	Aplikasi BFS dan DFS	12
3.1	Pendekatan Umum	12
3.2	Parsing	12
3.2.1	Pseudocode Algoritma Parsing	13
3.2.2	Jenis File HTML yang Dapat Dimasukkan	14
3.2.3	Contoh Masukan dan Keluaran Parsing	15
3.3	Selector	16
3.3.1	Selector Legal dan Tidak Legal	17
3.3.2	Pemrosesan Tiap Selector	17
3.3.3	Contoh Penggunaan Selector pada File HTML	19
3.4	Traversal	19
3.4.1	Pseudocode BFS dan DFS	20
3.4.2	Contoh Traversal pada Pohon DOM	22
4	Implementasi dan Pengujian	26
4.1	Lingkungan dan Struktur Implementasi	26
4.2	Implementasi Backend	27
4.2.1	Struktur Data DOM	27
4.2.2	Modul Parser dan Scraper	27
4.2.3	Modul Selector	27
4.2.4	Modul Traversal	28
4.2.5	HTTP API	28

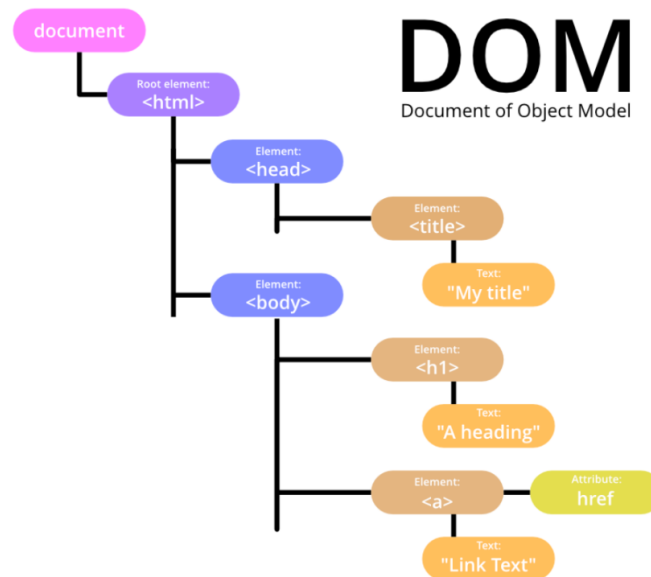


4.3	Implementasi Frontend	29
4.3.1	Form Input	30
4.3.2	Integrasi API	30
4.3.3	Visualisasi Pohon dan Log	30
4.3.4	Panel Metrik	30
4.4	Cara Menjalankan Aplikasi	31
4.5	Rencana dan Template Pengujian	31
4.5.1	Perintah Pengujian	31
4.5.2	Template Unit Test Backend	32
4.5.3	Template Pengujian API	32
4.5.4	Template Pengujian Fungsional Frontend	32
4.5.5	Template Hasil Eksperimen BFS dan DFS	33
4.5.6	Placeholder Analisis Hasil	33
5	Kesimpulan dan Saran	34
5.1	Kesimpulan	34
5.2	Saran	35
6	Lampiran	36
6.1	Tabel Checklist	36
6.2	Tautan GitHub	36
6.3	Web Deploy (via Azure)	36

Bab 1

Pendahuluan

1.1 Deskripsi Tugas



Gambar 1.1: Struktur Pohon DOM

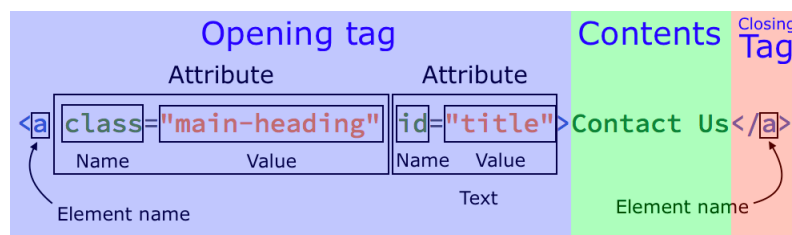
Document Object Model (DOM) merupakan representasi terstruktur dari dokumen HTML dalam bentuk tree yang memungkinkan setiap elemen pada halaman web diakses, dimodifikasi, dan dimanipulasi oleh program. Struktur ini merepresentasikan hubungan antar elemen HTML seperti parent, child, dan sibling sehingga memudahkan pengolahan dokumen secara terprogram. Struktur sebuah file HTML pada umumnya meliputi elemen root `<html>` yang berisi dua bagian utama yaitu `<head>` dan `<body>`. Bagian `<head>` berisi metadata seperti judul halaman, link stylesheet, dan script, sedangkan `<body>` berisi konten utama halaman seperti teks, gambar, tombol, dan elemen HTML lainnya yang ditampilkan kepada pengguna.

Sebuah website menampilkan sebuah DOM melalui sebuah file bernama Cascading Style Sheets (CSS) yang bertujuan mendeklarasikan visualisasi elemen-elemen

pada pohon. Deklarasi visualisasi ditempelkan pada elemen-elemen yang berkaitan melalui CSS Selector. Pada Tugas Besar 2 Strategi Algoritma ini, mahasiswa diminta untuk menelusuri dan mencari elemen pada struktur DOM berdasarkan CSS Selector tertentu dengan menggunakan algoritma penelusuran graf yaitu Breadth First Search (BFS) dan Depth First Search (DFS). Algoritma tersebut digunakan untuk menjelajahi struktur pohon DOM guna menemukan elemen yang sesuai dengan selector yang diberikan.

Komponen-komponen penting yang terdapat pada tugas besar ini antara lain:

1. **Tags** pada HTML merupakan penanda (markup) yang digunakan untuk mendefinisikan struktur dan jenis elemen dalam sebuah dokumen HTML. Tag memberi tahu browser bagaimana suatu bagian konten harus ditafsirkan dan ditampilkan pada halaman web.

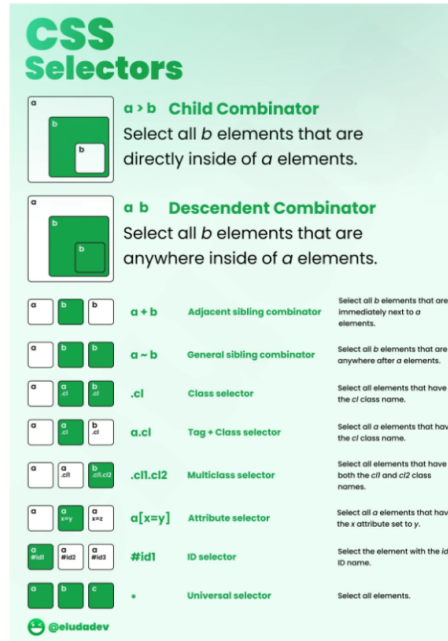


Gambar 1.2: Struktur Pohon DOM

Secara umum, tag HTML ditulis menggunakan tanda kurung sudut (<>). Sebagian besar elemen HTML memiliki pasangan tag pembuka dan tag penutup. Tag pembuka menandai awal suatu elemen, sedangkan tag penutup menandai akhir elemen tersebut. Tag penutup ditulis dengan menambahkan tanda garis miring/sebelum nama tag. Terdapat beberapa jenis tag HTML yang self-closing.

Dalam struktur DOM, setiap tag HTML direpresentasikan sebagai node pada pohon DOM. Hubungan antar tag membentuk struktur hierarki seperti parent, child, dan sibling. Struktur inilah yang memungkinkan dokumen HTML diproses sebagai sebuah pohon ketika dilakukan penelusuran menggunakan algoritma seperti BFS atau DFS.

2. **CSS Selector** adalah mekanisme pada CSS untuk memilih elemen HTML tertentu pada struktur DOM agar dapat diberikan aturan style. Dengan selector, kita dapat menentukan elemen mana yang akan dikenai aturan visual seperti warna, ukuran, atau tata letak.



Gambar 1.3: Struktur Pohon DOM

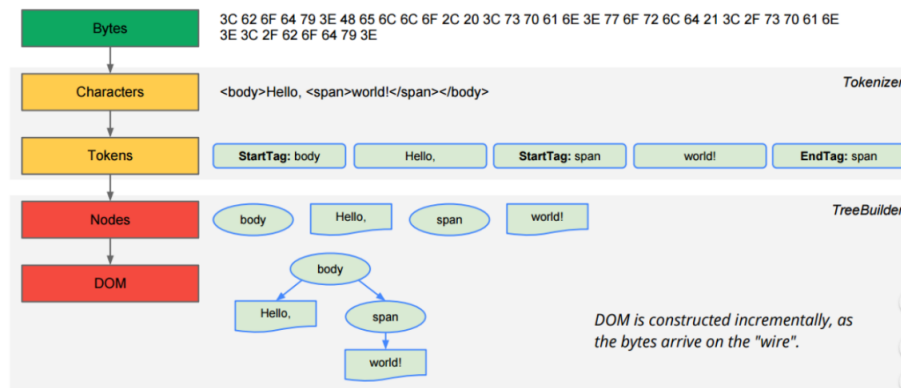
Beberapa selector dasar yang umum digunakan adalah sebagai berikut.

- Tag Selector.** Memilih elemen berdasarkan nama tag HTML. Contoh: `p` memilih semua elemen `<p>`.
- Class Selector.** Memilih elemen yang memiliki atribut class tertentu, ditulis dengan tanda titik (`.`). Contoh: `.box`.
- ID Selector.** Memilih elemen dengan atribut id tertentu, ditulis dengan tanda pagar (`#`). Contoh: `#header`
- Universal Selector.** Memilih semua elemen HTML. Contoh: `*`.

Combinator merupakan cara untuk menggabungkan beberapa jenis selector dalam penentuan elemen yang akan terpengaruh, dan terdiri dari list berikut ini.

- Child Combinator (`>`)
- Descendent Combinator (`" "`)
- Adjacent Sibling Combinator (`+`)
- General Sibling Combinator (`~`)

- HTML Parsing** adalah Proses membaca dokumen HTML dan mengubahnya menjadi struktur data pohon (tree) yang merepresentasikan hubungan antar elemen HTML. Pada proses ini, parser memproses dokumen HTML secara berurutan, mengenali tag pembuka dan tag penutup, kemudian menyusunnya ke dalam struktur hierarki.



Gambar 1.4: Struktur Pohon DOM

Setiap tag HTML direpresentasikan sebagai node pada pohon. Tag yang berada di dalam tag lain akan menjadi child node, sedangkan tag yang membungkusnya menjadi parent node. Node paling atas pada struktur ini disebut root, yang umumnya merupakan elemen `<html>`. Dari node root tersebut, elemen-elemen lain seperti `<head>` dan `<body>` akan menjadi cabang yang membentuk struktur pohon DOM.

Hasil dari proses parsing adalah DOM Tree, yaitu representasi pohon dari dokumen HTML yang memungkinkan elemen-elemen pada halaman web ditelusuri, diakses, dan dimanipulasi secara terstruktur. Struktur pohon ini kemudian dapat digunakan dalam proses penelusuran elemen, misalnya dengan menggunakan algoritma Breadth First Search (BFS) atau Depth First Search (DFS).

1.2 Spesifikasi Wajib

Pada tugas ini, mahasiswa diminta untuk membuat aplikasi traversal pohon HTML (DOM Tree) yang menggunakan algoritma Breadth First Search (BFS) dan Depth First Search (DFS) untuk melakukan pencarian elemen berdasarkan CSS selector pada struktur DOM.

- Buatlah sebuah aplikasi berbasis web yang terdiri dari frontend bebas dan backend dalam bahasa Go, C#, Rust, atau Typescript yang mengimplementasikan algoritma BFS dan DFS untuk melakukan penelusuran CSS pada pohon DOM.
- Aplikasi dapat melakukan scraping HTML dari sebuah website melalui URL yang diberikan oleh pengguna.
- Secara umum, berikut adalah input yang diharapkan.
 - URL website atau memasuki teks HTML sendiri
 - Pilihan algoritma traversal (BFS / DFS)
 - CSS selector

- Jumlah hasil yang diinginkan:
 - * Top n kemunculan
 - * Semua kemunculan
- HTML yang diperoleh kemudian diparsing menjadi struktur data pohon (DOM Tree).
- Output yang diharapkan adalah sebagai berikut.
 1. Aplikasi dapat menampilkan visualisasi struktur pohon DOM, disertai dengan keterangan kedalaman maksimum tree.
 2. Aplikasi dapat menampilkan highlight pada jalur yang di-traversal oleh algoritma penelusuran yang dipilih untuk menandakan elemen-elemen yang terpengaruh.
 3. Aplikasi dapat menampilkan waktu pencarian serta banyak node yang dikunjungi.
 4. Setelah melakukan penelusuran, aplikasi menyimpan sebuah traversal log yang memiliki informasi tahapan penelusuran yang dilakukan.
- Repository bagian frontend dan backend boleh digabung/dipisah.
- Setiap kelompok harap mengisi nama kelompok dan anggotanya pada link berikut, dengan maksimal tanggal Minggu, 5 April 2026 23.59. Nama kelompok dilarang mengandung unsur SARA, penghinaan, provokasi, atau hal-hal lain yang bertentangan dengan norma hukum.
- Perlu diperhatikan bahwa kelompok tubes untuk mata kuliah ini tidak boleh sama. Jadi jika berkelompok dengan X pada tubes ini, maka pada tubes selanjutnya anda tidak diperkenankan berkelompok dengan X.

1.3 Spesifikasi Bonus

- **(4 poin) [Video]** Membuat video tentang aplikasi BFS dan DFS pada penelusuran pohon DOM kemudian mengunggahnya di Youtube. Video dibuat harus memiliki audio dan menampilkan wajah dari setiap anggota kelompok. Untuk contoh video tubes stima tahun-tahun sebelumnya dapat dilihat di Youtube dengan kata kunci “Tubes Stima”, “strategi algoritma”, “Tugas besar stima”, dll. Semakin menarik video, maka semakin banyak poin yang diberikan.
- **(5 poin) [Deploy]** Aplikasi web yang telah dibuat perlu di deploy menggunakan Microsoft Azure Virtual Machine (VM) sehingga aplikasi dapat diakses secara publik melalui internet. Sebagai mahasiswa, terdapat kredit gratis sebesar \$100 dari Microsoft untuk dapat digunakan hanya dengan menggunakan email kampus ITB. Karena layanan ini menggunakan sistem pembayaran berbasis penggunaan (pay-as-you-go), penggunaan Virtual Machine akan mengurangi kredit yang dimiliki. Oleh karena itu, disarankan untuk mematikan

(stop/deallocate) Virtual Machine ketika tidak digunakan agar kredit Azure tidak cepat habis.

- **(2 Poin) [Docker]** Aplikasi dijalankan menggunakan Docker baik untuk frontend maupun backend.
- **(6 Poin) [Animasi]** Menampilkan animasi penelusuran pohon DOM untuk segala pencarian.
- **(3 Poin) [Multithreading]** Mengimplementasikan multithreading pada algoritma BFS dan/atau DFS untuk melakukan traversal pohon DOM secara paralel. Proses traversal dibagi ke beberapa thread/worker sehingga beberapa node dapat diproses secara bersamaan.
- **(3 Poin) [LCA Binary Lifting]** Mengimplementasikan algoritma Lowest Common Ancestor (LCA) dengan teknik Binary Lifting pada pohon DOM yang dihasilkan dari proses parsing HTML. LCA digunakan untuk menentukan node leluhur terendah yang sama dari dua node dalam sebuah pohon. Dalam konteks DOM, fitur ini dapat digunakan untuk mengetahui elemen HTML terdekat yang menjadi parent bersama dari dua elemen tertentu. Penjelasan lebih lanjut dapat dilihat pada link berikut.

Bab 2

Dasar Teori

2.1 Document Object Model

Document Object Model (DOM) adalah representasi dokumen HTML dalam bentuk struktur pohon yang dapat diakses oleh program. Setiap elemen HTML direpresentasikan sebagai node, sedangkan hubungan antarelemen direpresentasikan sebagai relasi *parent*, *child*, dan *sibling*. Pada konteks tugas ini, DOM menjadi model data utama karena proses pencarian elemen berdasarkan CSS selector dapat dipandang sebagai penelusuran node pada pohon berakar [1].

Sebuah pohon DOM memiliki satu akar utama, umumnya elemen `html`. Di bawah akar tersebut terdapat subtree seperti `head` dan `body`, lalu elemen-elemen lain yang bersarang di dalamnya. Informasi yang penting untuk pencarian meliputi nama tag, atribut, teks, daftar anak, parent, dan kedalaman node. Kedalaman dipakai untuk mengetahui posisi node terhadap root, sedangkan jalur dari root ke node dapat digunakan untuk menampilkan rute hasil pencarian pada antarmuka aplikasi.

2.2 HTML Parsing

HTML parsing adalah proses mengubah dokumen HTML mentah menjadi struktur DOM yang terorganisasi. Parser membaca token HTML seperti *start tag*, *end tag*, teks, dan *self-closing tag*, lalu menyusun token tersebut menjadi pohon sesuai aturan penyarangan elemen HTML [2]. Dalam implementasi aplikasi, tahap ini diperlukan sebelum BFS atau DFS dijalankan karena algoritma traversal membutuhkan struktur node dan relasi parent-child yang eksplisit.

Salah satu pendekatan umum untuk membangun pohon dari token HTML adalah menggunakan stack. Ketika parser menemukan tag pembuka, node baru dibuat sebagai anak dari node pada puncak stack, kemudian node tersebut dimasukkan ke stack jika bukan elemen kosong. Ketika tag penutup ditemukan, stack dipop hingga tag yang sesuai ditemukan. Untuk tag kosong seperti `img`, `br`, `meta`, atau `input`, node tetap dimasukkan ke pohon tetapi tidak menjadi parent aktif. Pendekatan ini efisien karena dokumen dapat diproses secara linear terhadap panjang input.

2.3 CSS Selector

CSS selector adalah pola yang digunakan untuk memilih elemen HTML tertentu. Dalam CSS, selector berfungsi menghubungkan aturan gaya dengan elemen target; dalam aplikasi ini, selector digunakan sebagai predikat pencarian terhadap node DOM [3]. Sebuah node dianggap cocok apabila tag, atribut, atau relasi strukturalnya memenuhi selector yang diberikan pengguna.

Selector sederhana yang relevan dengan implementasi terdiri atas tag selector, class selector, id selector, dan universal selector. Tag selector memilih node berdasarkan nama tag, misalnya `article`. Class selector memilih node yang memiliki class tertentu, misalnya `.product_pod`. ID selector memilih node dengan atribut id tertentu, misalnya `#main`. Universal selector `*` memilih semua node.

Selain selector sederhana, CSS juga mengenal combinator untuk menyatakan relasi antarnode. Descendant combinator `A B` memilih node B yang memiliki ancestor A. Child combinator `A > B` memilih node B yang parent langsungnya adalah A. Adjacent sibling combinator `A + B` memilih node B yang muncul tepat setelah sibling A. General sibling combinator `A ~ B` memilih node B yang muncul setelah sibling A pada parent yang sama. Keempat relasi ini sesuai dengan informasi struktural yang tersedia pada pohon DOM.

2.4 Traversal Graf pada Pohon DOM

Pohon DOM dapat dipandang sebagai graf terhubung tanpa siklus dengan satu root. Oleh karena itu, pencarian elemen pada DOM dapat dilakukan dengan algoritma traversal graf. Dua strategi traversal yang digunakan pada tugas ini adalah Breadth-First Search (BFS) dan Depth-First Search (DFS). Keduanya mengunjungi node-node pada pohon dan mengevaluasi selector pada setiap node, tetapi memiliki urutan kunjungan yang berbeda [4].

2.4.1 Breadth-First Search

Breadth-First Search menelusuri pohon secara melebar dari root. Node pada kedalaman yang lebih kecil dikunjungi lebih dahulu sebelum node pada kedalaman berikutnya. Secara umum, BFS menggunakan struktur data queue dengan prinsip FIFO (*First In, First Out*). Pada DOM, BFS cocok digunakan ketika elemen target diperkirakan berada dekat dengan root atau ketika urutan hasil berdasarkan level pohon lebih diinginkan.

Jika jumlah node adalah N , kompleksitas waktu BFS adalah $O(N)$ karena setiap node dikunjungi paling banyak satu kali. Kompleksitas ruangnya bergantung pada lebar maksimum pohon, yaitu $O(W)$, karena queue dapat menyimpan banyak node pada level yang sama. Dalam pencarian berbasis selector, biaya total dapat ditulis sebagai $O(N \cdot C)$, dengan C sebagai biaya evaluasi selector pada satu node.

2.4.2 Depth-First Search

Depth-First Search menelusuri pohon secara mendalam. Algoritma ini mengunjungi satu cabang sampai sejauh mungkin, lalu kembali untuk memproses cabang lain. DFS dapat diimplementasikan secara rekursif atau iteratif menggunakan stack dengan prinsip LIFO (*Last In, First Out*). Pada DOM, DFS cocok untuk mengeksplorasi subtree yang dalam karena traversal segera masuk ke anak sebuah node sebelum berpindah ke sibling berikutnya.

Sama seperti BFS, kompleksitas waktu DFS adalah $O(N)$ untuk mengunjungi seluruh node. Kompleksitas ruang DFS iteratif bergantung pada tinggi pohon dan jumlah node yang tersimpan pada stack, sehingga pada pohon yang relatif seimbang dapat mendekati $O(H)$, dengan H sebagai tinggi pohon. Pada kasus terburuk, ruang yang digunakan tetap dapat mencapai $O(N)$.

2.5 Pencarian Elemen dan Metrik Traversal

Pencarian elemen pada aplikasi ini dilakukan dengan menggabungkan traversal dan selector matching. Traversal menentukan urutan node yang dikunjungi, sedangkan selector menentukan apakah node tersebut masuk ke daftar hasil. Karena BFS dan DFS memakai predikat selector yang sama, himpunan hasil untuk pencarian tanpa limit seharusnya sama; perbedaannya terletak pada urutan hasil, traversal log, dan jumlah node yang dikunjungi sebelum berhenti ketika fitur Top- N digunakan.

Beberapa metrik penting yang digunakan untuk mengevaluasi pencarian adalah jumlah node yang dikunjungi, jumlah node yang cocok, kedalaman maksimum yang dijangkau, waktu eksekusi, dan status berhenti karena limit. Jumlah node yang dikunjungi menunjukkan besar area DOM yang dieksplorasi. Kedalaman maksimum membantu menggambarkan seberapa jauh traversal masuk ke struktur dokumen. Waktu eksekusi digunakan untuk membandingkan efisiensi praktis BFS dan DFS pada selector serta dokumen yang sama. Traversal log menyimpan urutan kunjungan node sehingga proses pencarian dapat divisualisasikan kembali pada frontend.

2.6 Aplikasi Web dan Pertukaran Data

Aplikasi tugas besar ini memisahkan pemrosesan utama pada backend dan visualisasi pada frontend. Backend bertugas mengambil atau menerima HTML, melakukan parsing, menjalankan BFS atau DFS, mengevaluasi selector, dan mengembalikan hasil dalam format JSON. Frontend bertugas menyediakan masukan pengguna, menampilkan pohon DOM, menyorot hasil pencarian, serta menyajikan metrik dan log traversal.

Format JSON sesuai untuk pertukaran data karena struktur node, atribut, daftar anak, hasil pencarian, dan log traversal dapat direpresentasikan secara hierarkis. Dengan demikian, pohon DOM yang diproses di backend dapat divisualisasikan kembali oleh frontend tanpa kehilangan informasi penting seperti tag, atribut, teks, kedalaman, path node, dan status kecocokan selector.

Bab 3

Aplikasi BFS dan DFS

3.1 Pendekatan Umum

Pendekatan utama pada aplikasi ini berfokus pada dua strategi traversal pohon DOM, yaitu Breadth-First Search (BFS) dan Depth-First Search (DFS). Sebelum traversal dijalankan, masukan HTML terlebih dahulu diparse menjadi pohon DOM agar setiap elemen memiliki relasi parent-child yang jelas. Setelah struktur pohon terbentuk, selector CSS dipakai sebagai filter untuk menentukan node mana yang dianggap cocok selama proses penelusuran.

Pada BFS, penelusuran dilakukan per level menggunakan queue (FIFO), sehingga node yang lebih dekat ke akar diproses lebih dahulu. Karakter ini membuat BFS cocok ketika tujuan utama adalah menemukan elemen yang berada pada kedalaman dangkal atau ketika urutan hasil berdasarkan level lebih diutamakan. Selama traversal, setiap node yang dikunjungi tetap dievaluasi terhadap selector, sehingga hasil tidak hanya bergantung pada urutan kunjungan, tetapi juga pada kecocokan kriterianya.

Pada DFS, penelusuran dilakukan sedalam mungkin pada satu cabang menggunakan stack (LIFO), lalu melakukan backtrack ke cabang lain. Pendekatan ini cocok untuk eksplorasi struktur yang dalam karena sistem segera masuk ke subtree sebelum berpindah ke sibling berikutnya. Seperti pada BFS, selector tetap dievaluasi di setiap langkah agar hanya node yang memenuhi kriteria yang dicatat sebagai match.

Perbedaan paling mencolok dari BFS dan DFS pada implementasi ini terletak pada urutan kunjungan node, jejak traversal, serta kemungkinan waktu tercapainya limit hasil. Meskipun urutan temuan dapat berbeda, keduanya menggunakan aturan selector yang sama dan menghasilkan keluaran dalam format metrik yang sama, seperti jumlah node dikunjungi, kedalaman maksimum, dan durasi eksekusi.

3.2 Parsing

Parsing pada program ini bertujuan mengubah dokumen HTML mentah menjadi representasi pohon DOM yang dapat ditelusuri oleh BFS maupun DFS. Tahap ini menjadi fondasi karena kualitas hasil traversal sangat bergantung pada ketepatan

struktur parent-child yang dibentuk saat parsing. Implementasi di backend menggunakan tokenizer HTML berbasis stream dari pustaka `golang.org/x/net/html`, sehingga dokumen dibaca token per token tanpa perlu membangun representasi string antara yang besar di memori.

Pendekatan yang dipakai adalah kombinasi *token stream processing* dan *stack-based tree construction*. Program membuat node akar semu `document` dengan kedalaman `-1`, lalu memelihara stack elemen aktif untuk menentukan parent dari token yang sedang diproses. Ketika menemukan `StartTagToken`, parser membuat node baru, menyalin atribut dari tokenizer, menautkan node tersebut ke parent pada puncak stack, dan mendorongnya ke stack jika tag bukan *self-closing*. Ketika menemukan `TextToken`, isi teks dibersihkan dengan `TrimSpace`. Jika tidak kosong, teks disimpan pada node yang sedang aktif. Ketika menemukan `EndTagToken`, stack dipop sampai tag yang sesuai ditemukan sehingga parser tetap toleran terhadap struktur penutup yang tidak rapi. Untuk `SelfClosingTagToken`, node tetap dibuat dan ditautkan ke parent, tetapi tidak didorong ke stack. Saat `ErrorToken` mencapai EOF, parser mengembalikan elemen akar HTML pertama sebagai root hasil parsing.

3.2.1 Pseudocode Algoritma Parsing

Algorithm 1: ParseHTML

```
Require: rawHTML berupa string HTML
Ensure: rootNode atau error
1: tokenizer  $\leftarrow$  NEWHTMLTOKENIZER(rawHTML)
2: sentinel  $\leftarrow$  Node(tag = "document", depth = -1)
3: stack  $\leftarrow$  [sentinel]
4: while true do
5:   tokenType  $\leftarrow$  tokenizer.Next()
6:   if tokenType = ErrorToken then
7:     if tokenizer.Err() = EOF then
8:       if sentinel.children tidak kosong then
9:         return sentinel.children[0]
10:      else
11:        return error("empty document")
12:      end if
13:    end if
14:   else if tokenType = StartTagToken then
15:     tag, hasAttr  $\leftarrow$  tokenizer.TagName()
16:     attrs  $\leftarrow$  EXTRACTATTRIBUTES(tokenizer, hasAttr)
17:     parent  $\leftarrow$  top(stack)
18:     node  $\leftarrow$  Node(tag = tag, attrs = attrs, parent = parent, depth =
|stack| - 1)
19:     tambahkan node ke parent.children
20:     if tag bukan anggota SelfClosingTagSet then
21:       push(stack, node)
22:     end if
```

```
23:   else if tokenType = TextToken then
24:       txt ← TRIMSPACE(tokenizer.Text())
25:       if txt ≠ "" then
26:           top(stack).text ← txt
27:       end if
28:   else if tokenType = EndTagToken then
29:       closingTag ← tokenizer.TagName()
30:       while |stack| > 1 do
31:           topNode ← pop(stack)
32:           if topNode.tag = closingTag then
33:               break
34:           end if
35:       end while
36:   else if tokenType = SelfClosingTagToken then
37:       tag, hasAttr ← tokenizer.TagName()
38:       attrs ← EXTRACTATTRIBUTES(tokenizer, hasAttr)
39:       parent ← top(stack)
40:       node ← Node(tag = tag, attrs = attrs, parent = parent, depth =
|stack| - 1)
41:       tambahkan node ke parent.children
42:   end if
43: end while
```

Algorithm 2: ExtractAttributes

Require: tokenizer, hasAttr

Ensure: attrs berupa map key-value atribut

```
1: attrs ← map kosong
2: if hasAttr = false then
3:     return attrs
4: end if
5: repeat
6:     key, value, more ← tokenizer.TagAttr()
7:     attrs[key] ← value
8: until more = false
9: return attrs
```

3.2.2 Jenis File HTML yang Dapat Dimasukkan

Program menerima masukan HTML dari tiga sumber, yaitu URL (melalui HTTP GET), berkas lokal, dan string HTML manual. Untuk berkas lokal, ekstensi yang umum seperti `.html` dan `.htm` dapat diproses selama kontennya valid sebagai dokumen HTML teks. Karena parsing menggunakan tokenizer HTML5 yang toleran, dokumen dengan struktur tidak sepenuhnya rapi seperti tag yang tidak tertutup sempurna tetap dapat diparse dalam banyak kasus. Implementasi juga mendukung elemen *void/self-closing* populer seperti `img`, `br`, `hr`, `input`, `meta`, `link`, dan sejenisnya. Sebaliknya, parser ini tidak menargetkan semantik XML ketat atau kebutuhan

namespace khusus, sehingga fokusnya tetap pada HTML web umum untuk kebutuhan traversal selector.

3.2.3 Contoh Masukan dan Keluaran Parsing

Contoh pertama memperlihatkan masukan sederhana dengan elemen bertingkat dan atribut. Setelah diparse, keluaran berupa node root `html` beserta subtree lengkapnya.

Contoh 1: Input dan Output Parsing

```
// INPUT HTML
<html>
<body id="main">
<section class="catalog">
<article class="item featured">Alpha</article>
<article class="item">Beta</article>
</section>
</body>
</html>

// OUTPUT
{
  "tag": "html",
  "depth": 0,
  "children": [
    {
      "tag": "body",
      "attributes": {"id": "main"},
      "depth": 1,
      "children": [
        {
          "tag": "section",
          "attributes": {"class": "catalog"},
          "depth": 2,
          "children": [
            {
              "tag": "article",
              "attributes": {"class": "item featured"},
              "text": "Alpha",
              "depth": 3
            },
            {
              "tag": "article",
              "attributes": {"class": "item"},
              "text": "Beta",
              "depth": 3
            }
          ]
        }
      ]
    }
  ]
}
```



```
]
}
]
}
]
}
```

Contoh kedua menekankan penanganan elemen *self-closing*. Elemen `img` tetap masuk sebagai node anak, tetapi tidak mengubah stack kedalaman karena tidak memiliki pasangan `EndTag`.

Contoh 2: Self-Closing Tag

```
// INPUT HTML
<html>
<body>
<div>
<img class="hero" />
<p id="lead">Intro</p>
</div>
</body>
</html>

// OUTPUT
html(depth=0)
body(depth=1)
div(depth=2)
img[class="hero"] (depth=3)
p[id="lead"] text="Intro" (depth=3)
```

Dengan pendekatan ini, hasil parsing konsisten untuk dipakai pada tahap selector matching dan traversal BFS/DFS, karena struktur pohon, atribut, teks, dan kedalaman node sudah tersedia secara eksplisit sejak awal proses.

3.3 Selector

Modul selector bertanggung jawab untuk menentukan apakah sebuah node DOM cocok dengan pola CSS selector yang dimasukkan pengguna. Secara implementasi, proses seleksi dibagi menjadi dua lapisan. Lapisan pertama adalah `MatchSelector`, yaitu pencocokan selector sederhana pada satu node. Lapisan kedua adalah `Match`, yaitu evaluasi selector yang melibatkan relasi antarnode seperti parent-child atau sibling. Pemisahan ini membuat proses lebih terstruktur karena pengecekan atribut node dan pengecekan relasi pohon tidak tercampur pada satu fungsi.

Pada awal proses, selector dinormalisasi dengan menambahkan spasi di sekitar operator `>`, `+`, dan `~`, lalu multiple space diringkas menjadi satu spasi. Tujuannya agar bentuk input seperti `div>p`, `div > p`, atau `div > p` tetap diperlakukan sama. Setelah normalisasi, modul melakukan pencarian combinator dan mengevaluasi sele-

ctor secara rekursif dari sisi kanan ke kiri agar node saat ini selalu diuji pada bagian selector yang paling dekat dengan dirinya.

3.3.1 Selector Legal dan Tidak Legal

Selector yang legal adalah selector yang benar-benar didukung oleh implementasi fungsi `MatchSelector` dan `Match`. Pada implementasi, selector sederhana yang legal mencakup tag selector, class selector tunggal, id selector tunggal, dan universal selector. Selain itu, selector relasional dengan combinator `space` (descendant), `>` (child), `+` (adjacent sibling), dan `~` (general sibling) juga legal selama setiap operand kiri dan kanannya berupa selector sederhana yang didukung.

Contoh Selector Legal

```
tag selector: article, p, div
class selector: .card, .featured
id selector: #main, #news
universal selector: *
descendant combinator: section article
child combinator: section > article
adjacent sibling: article + p
general sibling: article ~ p
```

Selector tidak legal adalah selector CSS yang tidak memiliki dukungan eksplisit dalam kode. Jika selector seperti ini diberikan, sistem tidak melakukan parsing tingkat lanjut, melainkan mencoba mencocokkan string apa adanya pada jalur fallback, sehingga umumnya menghasilkan tidak ada match.

Contoh Selector Tidak Legal (Tidak Didukung)

```
selector gabungan satu token: article.card, div#main
attribute selector: [class="card"], [id]
pseudo-class: p:first-child, li:nth-child(2)
pseudo-element: p::before
selector list: article, p
namespace selector: svg|rect
```

3.3.2 Pemrosesan Tiap Selector

Pemrosesan tag selector dilakukan dengan membandingkan langsung `node`. Tag terhadap string selector. Pemrosesan class selector dilakukan ketika token diawali titik; nilai atribut `class` dipecah berdasarkan spasi, lalu diperiksa apakah nama class target muncul sebagai token utuh. Pemrosesan id selector dilakukan ketika token diawali tanda pagar dan dibandingkan langsung dengan atribut `id`. Universal selector `*` selalu bernilai benar untuk node apa pun.

Untuk combinator, pemrosesan dilakukan pada fungsi `Match`. Combinator `>` memeriksa apakah node saat ini cocok dengan operand kanan dan parent langsungnya cocok dengan operand kiri. Combinator `+` memeriksa sibling elemen sebelumnya

yang tepat satu posisi sebelum node. Combinator $\sim$ memeriksa apakah ada sibling sebelumnya yang cocok. Adapun combinator spasi memeriksa apakah ada ancestor mana pun yang cocok dengan selector sisi kiri. Dengan pendekatan ini, selector relational dapat dievaluasi secara rekursif terhadap struktur DOM yang sudah dibentuk pada tahap parsing.

Algorithm 3: MatchSelector

Require: node, selector

Ensure: nilai boolean cocok/tidak cocok

```
1: if selector = "*" then
2:   return true
3: end if
4: if selector diawali "#" then
5:   return node.attributes["id"] = selector[1:]
6: end if
7: if selector diawali "." then
8:   classes  $\leftarrow$  split spasi dari node.attributes["class"]
9:   return selector[1:] ada di classes
10: end if
11: return node.tag = selector
```

Algorithm 4: Match

Require: node, selector

Ensure: nilai boolean cocok/tidak cocok

```
1: normalisasi spasi dan operator pada selector
2: if ada operator " > " pada selector then
3:   pecah menjadi left dan right
4:   return MatchSelector(node, right)
   AND node.parent != nil AND Match(node.parent, left)
5: end if
6: if ada operator " + " pada selector then
7:   pecah menjadi left dan right
8:   if MatchSelector(node, right) = false then
9:     return false
10:  end if
11:  prev  $\leftarrow$  sibling elemen tepat sebelum node
12:  return prev != nil AND Match(prev, left)
13: end if
14: if ada operator " ~ " pada selector then
15:   pecah menjadi left dan right
16:   if MatchSelector(node, right) = false then
17:     return false
18:   end if
19:   return ada sibling sebelumnya yang memenuhi Match(sibling, left)
20: end if
21: if selector mengandung spasi tanpa operator lain then
```

```
22:   pecah jadi left (ancestor pattern) dan right (node pattern)
23:   if MatchSelector(node, right) = false then
24:       return false
25:   end if
26:   return ada ancestor yang memenuhi Match(ancestor, left)
27: end if
28: return MatchSelector(node, selector)
```

3.3.3 Contoh Penggunaan Selector pada File HTML

Contoh berikut memperlihatkan satu file HTML sederhana dan beberapa selector legal beserta hasil yang didapatkan.

File HTML Contoh

```
<html>
<body>
<section id="news">
<article class="card">Alpha</article>
<article class="card featured">
<p class="lead">Intro</p>
</article>
<p>Outside</p>
</section>
</body>
</html>
```

Pada file tersebut, selector **article** menghasilkan dua node karena ada dua elemen **article**. Selector **.card** juga menghasilkan dua node karena kedua **article** memiliki class **card**. Selector **#news** hanya menghasilkan satu node, yaitu elemen **section** yang memiliki id tersebut. Selector relasional **section > article** menghasilkan dua node karena kedua **article** adalah child langsung dari **section**. Selector **section article** juga menghasilkan dua node, tetapi mekanisme pencariannya berbasis relasi ancestor-descendant. Selector **article > p** menghasilkan satu node, yaitu **p** dengan class **lead**, sedangkan selector **article + p** dan **article ~ p** sama-sama menghasilkan satu node **p** terakhir karena node tersebut berada sesudah elemen **article** sebagai sibling pada parent yang sama.

Sebagai pembandingan, jika pada file yang sama digunakan selector tidak legal seperti **article.card** atau **p:first-child**, implementasi saat ini tidak memiliki parser khusus untuk bentuk tersebut sehingga hasilnya cenderung kosong. Oleh karena itu, untuk memperoleh hasil yang konsisten, pengguna harus menyusun query menggunakan bentuk selector yang memang didukung oleh modul selector.

3.4 Traversal

Traversal adalah tahap pencarian node pada pohon DOM yang sudah dihasilkan parser. Pada implementasi ini, traversal bekerja di atas struktur **Node** yang menyimpan

relasi parent-child, lalu mengunjungi node satu per satu sambil menjalankan selector matching. Program mendukung dua strategi, yaitu Breadth-First Search (BFS) dan Depth-First Search (DFS), dengan antarmuka yang sama melalui fungsi `SearchDOM`. Perbedaan keduanya bukan pada kriteria selector, tetapi pada urutan node yang dikunjungi.

Pada BFS, node dieksplorasi per level memakai queue (FIFO). Node root dikunjungi lebih dulu, lalu semua anak level berikutnya, dan seterusnya. Strategi ini cocok ketika elemen target cenderung berada di kedalaman yang dangkal. Pada DFS, node dieksplorasi sedalam mungkin di satu cabang memakai stack (LIFO), baru kemudian backtrack ke cabang lain. Pada kode program, urutan sibling kiri-ke-kanan tetap dipertahankan dengan mendorong anak ke stack dari kanan ke kiri.

Selain urutan eksplorasi, traversal juga menangani metrik dan kondisi berhenti dini. Setiap kunjungan menaikkan `VisitedCount`, memperbarui `MaxDepthVisited`, lalu memanggil `selector.Match`. Jika cocok, node disimpan ke `Matches` dengan metadata seperti `nodePath`, `depth`, dan `visitStep`. Jika `IncludeTraversalLog` aktif, event kunjungan juga dicatat. Ketika `Limit > 0` dan jumlah match telah mencapai limit, traversal berhenti lebih awal dengan `StoppedByLimit = true`.

3.4.1 Pseudocode BFS dan DFS

Algorithm 5: SearchDOM

Require: root, req berisi selector, algorithm, limit, opsi log

Ensure: SearchResult atau error

```
1: if root = nil then
2:   return error("root cannot be nil")
3: end if
4: if selector kosong then
5:   return error("selector cannot be empty")
6: end if
7: algo ← uppercase(trim(req.algorithm))
8: if algo kosong then
9:   algo ← BFS
10: end if
11: if algo = BFS then
12:   res ← SEARCHBFS(root, req)
13: else if algo = DFS then
14:   res ← SEARCHDFS(root, req)
15: else
16:   return error("unsupported algorithm")
17: end if
18: isi res.algorithmUsed dan res.selectorUsed
19: return res
```

Algorithm 6: searchBFS

Require: root, req

Ensure: res

```
1: inialisasi res, catat startTime
2: queue  $\leftarrow$  [root dengan nodePath = "0"]
3: while queue tidak kosong do
4:   current  $\leftarrow$  pop-front dari queue
5:   matched, shouldStop  $\leftarrow$  PROCESSVISIT(res, current, req)
6:   if shouldStop then
7:     break
8:   end if
9:   for setiap child dari current.node.Children dalam urutan asli do
10:    bentuk childPath dan pathFromTop
11:    append item anak ke belakang queue
12:  end for
13: end while
14: res.durationMs  $\leftarrow$  selisih waktu sekarang dengan startTime
15: return res
```

Algorithm 7: searchDFS

Require: root, req

Ensure: res

```
1: inialisasi res, catat startTime
2: stack  $\leftarrow$  [root dengan nodePath = "0"]
3: while stack tidak kosong do
4:   current  $\leftarrow$  pop-top dari stack
5:   matched, shouldStop  $\leftarrow$  PROCESSVISIT(res, current, req)
6:   if shouldStop then
7:     break
8:   end if
9:   for indeks anak i dari kanan ke kiri do
10:    bentuk childPath dan pathFromTop
11:    push item anak ke stack
12:  end for
13: end while
14: res.durationMs  $\leftarrow$  selisih waktu sekarang dengan startTime
15: return res
```

Algorithm 8: processVisit

Require: res, current, req

Ensure: (matched, shouldStop)

```
1: res.visitedCount  $\leftarrow$  res.visitedCount + 1
2: perbarui res.maxDepthVisited jika depth node saat ini lebih besar
3: matched  $\leftarrow$  MATCH(current.node, req.selector)
4: if matched then
```

```
5:   bentuk MatchHit dan append ke res.matches
6:   if req.includePathToMatch then
7:       salin pathFromTop ke hit.pathFromRoot
8:   end if
9: end if
10: if req.includeTraversalLog then
11:     append VisitEvent ke res.traversalLog
12: end if
13: if req.limit > 0 and jumlah match  $\geq$  limit then
14:     res.stoppedByLimit  $\leftarrow$  true
15:     return (matched, true)
16: end if
17: return (matched, false)
```

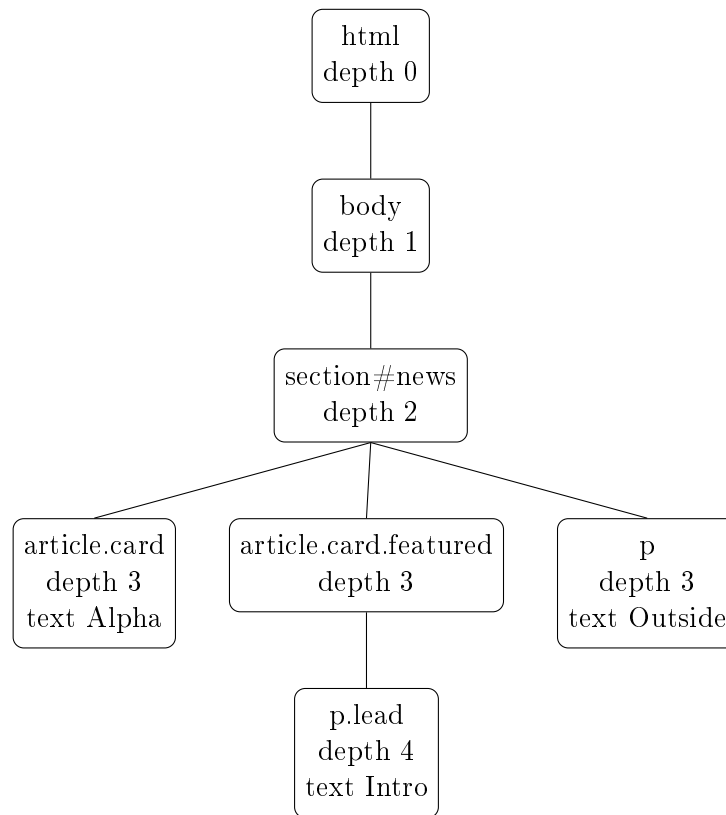
3.4.2 Contoh Traversal pada Pohon DOM

Contoh berikut menggunakan struktur DOM yang sama dengan section Selector agar perbandingan BFS dan DFS mudah diamati.

Input HTML

```
<html>
<body>
  <section id="news">
    <article class="card">Alpha</article>
    <article class="card featured">
      <p class="lead">Intro</p>
    </article>
    <p>Outside</p>
  </section>
</body>
</html>
```

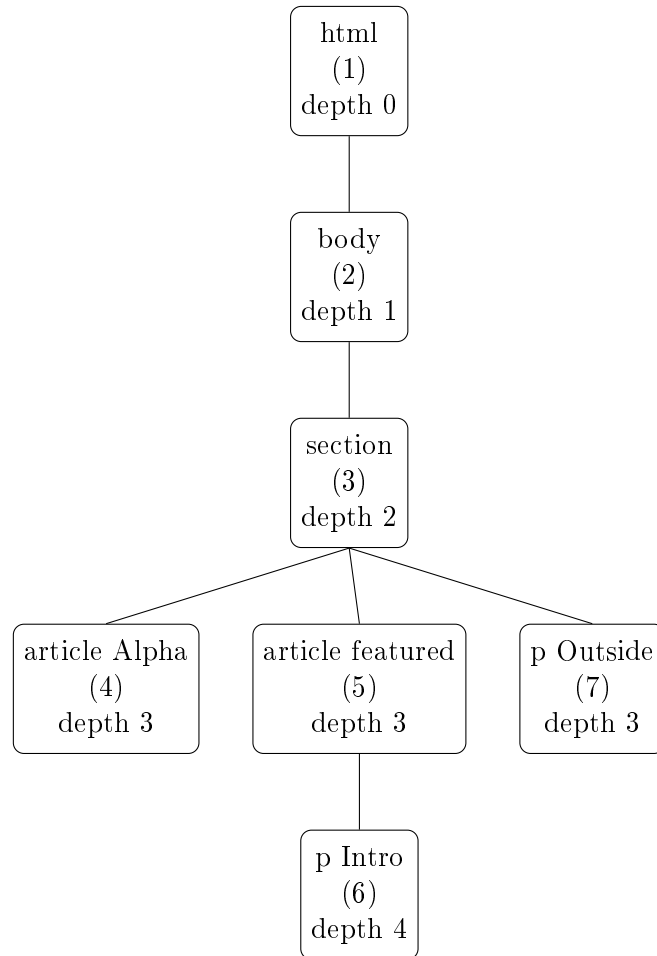
Pohon DOM Awal



Gambar 3.1: Pohon DOM hasil parsing

Hasil Eksplorasi BFS

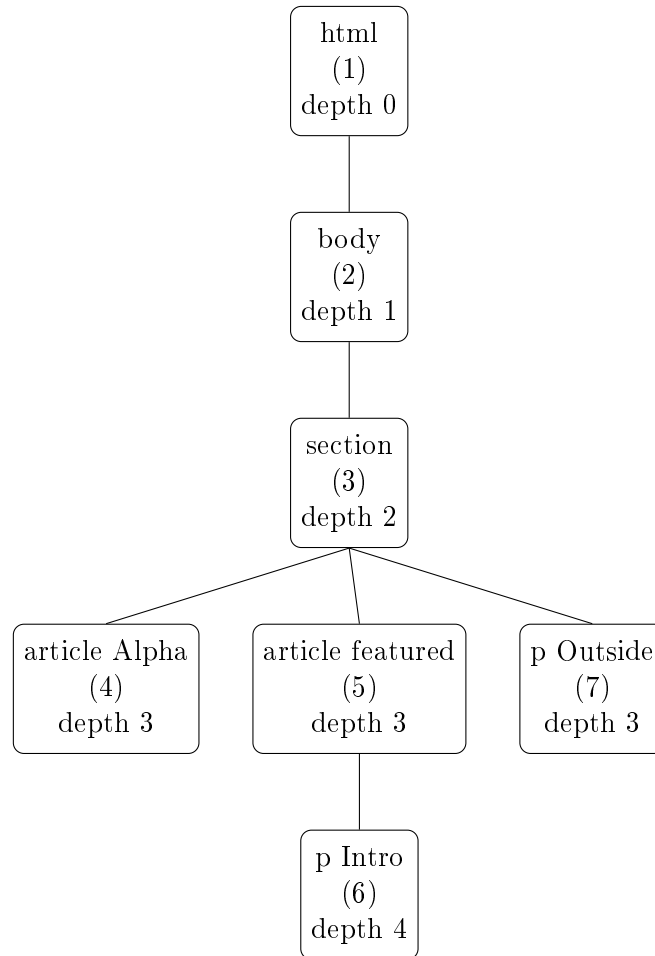
Untuk selector `p`, urutan kunjungan BFS pada pohon di atas adalah `html` $\rightarrow$ `body` $\rightarrow$ `section` $\rightarrow$ `article(Alpha)` $\rightarrow$ `article(featured)` $\rightarrow$ `p(Outside)` $\rightarrow$ `p(Intro)`.



Gambar 3.2: Pohon DOM dengan nomor urut eksplorasi BFS

Hasil Eksplorasi DFS

Untuk selector `p`, urutan kunjungan DFS (pre-order, left-to-right sesuai implementasi) adalah `html` $\rightarrow$ `body` $\rightarrow$ `section` $\rightarrow$ `article(Alpha)` $\rightarrow$ `article(featured)` $\rightarrow$ `p(Intro)` $\rightarrow$ `p(Outside)`.



Gambar 3.3: Pohon DOM dengan nomor urut eksplorasi DFS

Hasil akhir match selector **p** pada kedua algoritma tetap sama secara himpunan, yaitu dua node **p**. Perbedaannya berada pada urutan ditemukan, isi traversal log, dan potensi waktu berhenti jika **limit** diaktifkan. Contohnya, jika **limit** = 1, BFS pada pohon ini akan berhenti saat menemukan **p(Outside)**, sedangkan DFS berhenti saat menemukan **p(Intro)**.

Bab 4

Implementasi dan Pengujian

4.1 Lingkungan dan Struktur Implementasi

Implementasi aplikasi DOMTracer dibagi menjadi dua bagian utama, yaitu backend berbasis Go dan frontend berbasis React TypeScript. Backend bertanggung jawab atas pengambilan HTML, parsing DOM, pencocokan CSS selector, dan traversal BFS/DFS. Frontend bertanggung jawab atas antarmuka input, pengiriman request ke backend, visualisasi pohon DOM, panel metrik, dan traversal log.

Direktori/Berkas	Keterangan
backend/main.go	Entry point HTTP API yang terhubung dengan frontend melalui endpoint <code>/api/traverse</code> dan <code>/api/health</code> .
backend/cmd/server/main.go	Entry point server alternatif yang menyediakan endpoint <code>/api/scrape</code> , <code>/api/tree</code> , <code>/api/scrape-upload</code> , dan <code>/api/tree-upload</code> .
backend/cmd/cli/main.go	Program CLI untuk menjalankan pipeline HTML, parsing, traversal, dan penyimpanan traversal log.
backend/src/parser	Modul parser HTML dan scraper URL.
backend/src/selector	Modul pencocokan CSS selector sederhana dan combinator.
backend/src/traversal	Modul BFS, DFS, metrik traversal, hasil match, dan traversal log.
frontend/src	Modul API dan tipe data TypeScript yang digunakan frontend.
frontend/components	Komponen UI untuk form input, graf pohon, tree view, panel metrik, dan log traversal.
test	Kumpulan file HTML dan expected output yang disiapkan sebagai bahan pengujian.

Backend menggunakan modul Go `golang.org/x/net/html` untuk tokenisasi HTML. Frontend menggunakan Vite, React, TypeScript, Tailwind CSS, dan komponen UI pendukung. Komunikasi antara frontend dan backend dilakukan menggunakan HTTP request dengan payload dan response berformat JSON.

4.2 Implementasi Backend

4.2.1 Struktur Data DOM

Representasi DOM didefinisikan pada struct `Node` di modul `parser`. Setiap node menyimpan nama tag, teks, atribut, parent, children, dan depth. Field `Parent` diperlukan oleh selector combinator seperti `child`, `descendant`, `adjacent sibling`, dan `general sibling`. Field `Children` digunakan oleh algoritma BFS dan DFS untuk menelusuri subtree, sedangkan `Depth` digunakan untuk metrik dan visualisasi.

Ringkasan Struktur Node

```
type Node struct {  
    Tag      string  
    Text     string  
    Attributes map[string]string  
    Parent   *Node  
    Children []*Node  
    Depth    int  
}
```

4.2.2 Modul Parser dan Scraper

Modul `parser` memiliki dua fungsi utama, yaitu mengambil HTML dari URL dan mengubah HTML mentah menjadi pohon DOM. Fungsi `FetchHTML` membuat HTTP GET request dengan timeout 10 detik dan header `User-Agent` agar request lebih menyerupai browser. Fungsi `HTMLScraper` menggabungkan proses fetch dan parsing sehingga dapat langsung menghasilkan root DOM.

Parsing dilakukan oleh fungsi `ParseHTML`. Fungsi ini membuat tokenizer HTML, root semu `document`, dan stack node aktif. Ketika tokenizer membaca `StartTagToken`, parser membuat node baru, menyalin atribut, menautkannya ke parent pada puncak stack, lalu memasukkannya ke stack apabila tag tersebut bukan tag kosong. Ketika tokenizer membaca `TextToken`, teks dibersihkan dengan `TrimSpace` dan disimpan pada node aktif. Ketika tokenizer membaca `EndTagToken`, stack dipop sampai tag yang sesuai ditemukan. Untuk `SelfClosingTagToken`, node ditambahkan sebagai child tetapi tidak dimasukkan ke stack.

Daftar tag kosong seperti `img`, `br`, `hr`, `input`, `meta`, dan `link` disimpan dalam `selfClosingTags`. Dengan pendekatan ini, parser dapat membentuk relasi parent-child secara linear terhadap token HTML.

4.2.3 Modul Selector

Modul `selector` menyediakan dua tingkat pencocokan. Fungsi `MatchSelector` menangani selector sederhana, yaitu universal selector `*`, id selector `#id`, class selector `.class`, dan tag selector. Untuk class selector, nilai atribut `class` dipisahkan menggunakan `strings.Fields` sehingga pencocokan dilakukan terhadap token class utuh.

Fungsi **Match** menangani selector relasional. Sebelum diproses, string selector dinormalisasi dengan menambahkan spasi di sekitar operator `>`, `+`, dan `~`, kemudian spasi berlebih diringkas. Setelah itu, selector dievaluasi dari sisi kanan karena node yang sedang dikunjungi adalah kandidat hasil akhir. Implementasi mendukung:

- `A B`, untuk descendant combinator melalui pengecekan ancestor.
- `A > B`, untuk child combinator melalui pengecekan parent langsung.
- `A + B`, untuk adjacent sibling combinator melalui pengecekan sibling sebelumnya.
- `A ~ B`, untuk general sibling combinator melalui pengecekan semua sibling sebelumnya.

Dengan pemisahan ini, traversal tidak perlu mengetahui detail CSS selector. Traversal hanya memanggil `selector.Match(node, selector)` pada setiap node yang dikunjungi.

4.2.4 Modul Traversal

Modul **traversal** menyediakan fungsi utama **SearchDOM**. Fungsi ini menerima root DOM dan **SearchRequest**, memvalidasi selector, menentukan algoritma, lalu memanggil **searchBFS** atau **searchDFS**. Jika algoritma tidak diisi, sistem menggunakan BFS sebagai default.

Pada BFS, struktur data yang digunakan adalah queue. Root dimasukkan terlebih dahulu dengan path 0. Setiap node diproses dari depan queue, kemudian seluruh anaknya dimasukkan ke belakang queue sesuai urutan asli. Pada DFS, struktur data yang digunakan adalah stack. Anak node dimasukkan dari kanan ke kiri agar ketika dipop, urutan kunjungan tetap kiri ke kanan.

Kedua algoritma menggunakan fungsi bersama **processVisit**. Fungsi ini memperbarui **VisitedCount**, **MaxDepthVisited**, mengevaluasi selector, menyimpan **MatchHit**, menambahkan **VisitEvent** jika log diminta, dan menghentikan traversal jika jumlah match sudah mencapai limit **Top-N**.

Field Hasil	Makna
Matches	Daftar node yang cocok dengan selector.
VisitedCount	Jumlah node yang sudah dikunjungi traversal.
MaxDepthVisited	Kedalaman maksimum yang dicapai saat traversal.
DurationMs	Durasi traversal dalam milidetik.
TraversalLog	Daftar event kunjungan node per langkah.
StoppedByLimit	Bernilai true jika traversal berhenti karena Top-N .

4.2.5 HTTP API

Untuk integrasi dengan frontend, server pada **backend/main.go** menyediakan endpoint **POST /api/traverse**. Endpoint ini menerima input berupa URL atau HTML mentah, mode input, algoritma, CSS selector, dan limit hasil. Setelah menerima request, server menjalankan pipeline berikut:

1. Validasi field input dan `cssSelector`.
2. Jika `inputMode = html`, input langsung diparse dengan `ParseHTML`; selain itu input dianggap sebagai URL dan diproses dengan `HTMLScraper`.
3. Jalankan `SearchDOM` dengan traversal log selalu diaktifkan untuk kebutuhan visualisasi.
4. Bentuk set `traversedPaths` dan `matchedPaths` dari hasil traversal.
5. Konversi pohon `parser.Node` menjadi `treeNodeJSON` yang sudah memiliki `isTraversed`, `isMatched`, dan `nodeId`.
6. Konversi traversal log backend menjadi format log frontend.
7. Kirim response JSON berisi tree, traversal log, metrik, algoritma, dan selector.

Contoh Request POST `/api/traverse`

```
{
  "input": "https://example.com",
  "inputMode": "url",
  "algorithm": "BFS",
  "cssSelector": "a",
  "limit": 10
}
```

Ringkasan Response Frontend

```
{
  "tree": { "...": "annotated DOM tree" },
  "traversalLog": [],
  "metrics": {
    "searchTimeMs": 0,
    "visitedNodeCount": 0,
    "matchedNodeCount": 0,
    "maxDepth": 0
  },
  "algorithm": "BFS",
  "selector": "a"
}
```

Selain endpoint utama tersebut, server alternatif pada `backend/cmd/server/main.go` menyediakan endpoint yang lebih eksplisit untuk kebutuhan scraping, tree, dan upload file. Endpoint ini berguna apabila pengujian backend ingin dilakukan langsung menggunakan `curl` atau tool API tanpa frontend.

4.3 Implementasi Frontend

Frontend dibangun sebagai aplikasi satu halaman. Komponen `Index.tsx` menjadi halaman utama yang mengatur state hasil traversal, status loading, error,

tab aktif, dan status koneksi backend. Saat halaman dibuka, frontend memanggil `/api/health` untuk memastikan backend aktif.

4.3.1 Form Input

Komponen `InputForm.tsx` menyediakan konfigurasi traversal. Pengguna dapat memilih sumber input berupa URL, raw HTML, atau file HTML. Untuk mode file, browser membaca file dengan `FileReader` lalu mengirim isi file sebagai input HTML. Pengguna juga memilih algoritma BFS/DFS, CSS selector, dan batas hasil berupa semua hasil atau Top- N .

State form dikirim ke fungsi `runTraversal` pada `frontend/src/api.ts`. Fungsi ini membentuk payload `TraverseRequest`. Jika mode input adalah file, frontend mengirimnya sebagai mode `html` karena isi file sudah dibaca menjadi string.

4.3.2 Integrasi API

Modul `api.ts` menggunakan environment variable `VITE_API_BASE_URL`; jika tidak tersedia, default-nya adalah `http://localhost:8080`. Fungsi `runTraversal` mengirim request ke `/api/traverse`. Jika backend tidak dapat dihubungi, frontend menampilkan pesan error agar pengguna mengetahui bahwa server perlu dijalankan.

Tipe data response didefinisikan pada `frontend/src/types.ts`. Interface `ApiResponse` berisi `tree`, `traversalLog`, `metrics`, `algorithm`, dan `selector`. Shape ini disamakan dengan response dari `backend/main.go`.

4.3.3 Visualisasi Pohon dan Log

Komponen `TreeGraph.tsx` menampilkan DOM sebagai graf SVG. Layout pohon dihitung berdasarkan jumlah leaf pada setiap subtree. Setiap node memiliki koordinat x dan y , lalu edge digambar sebagai kurva antar parent dan child. Node yang cocok dengan selector diberi warna berbeda melalui field `isMatched`.

Komponen `TreeNode.tsx` menampilkan struktur DOM dalam bentuk tree view yang dapat dibuka dan ditutup. Node yang sudah dikunjungi diberi style berbeda melalui `isTraversed`, sedangkan node yang cocok diberi penanda match. Komponen ini juga menampilkan sebagian atribut dan potongan teks node.

Komponen `TraversalLog.tsx` menampilkan setiap langkah traversal. Log berisi nomor step, tag node, path node, pesan kunjungan, dan status match. Komponen ini juga menyediakan filter untuk menampilkan hanya node yang match.

4.3.4 Panel Metrik

Komponen `MetricsPanel.tsx` menampilkan empat metrik utama, yaitu waktu pencarian, jumlah node yang dikunjungi, jumlah match, dan kedalaman maksimum. Metrik ini berasal dari hasil traversal backend dan digunakan untuk membandingkan perilaku BFS dan DFS pada input yang sama.

4.4 Cara Menjalankan Aplikasi

Backend yang sesuai dengan endpoint frontend dapat dijalankan dari direktori `backend` dengan perintah berikut.

Menjalankan Backend Utama

```
go run ./main.go
```

Frontend dapat dijalankan dari direktori `frontend` dengan perintah berikut.

Menjalankan Frontend

```
bun run dev
```

Apabila ingin menjalankan server alternatif yang menyediakan endpoint `/api/scrape` dan `/api/tree`, perintah yang digunakan adalah sebagai berikut.

Menjalankan Server Alternatif

```
go run ./cmd/server/main.go
```

4.5 Rencana dan Template Pengujian

Pengujian akhir belum dilakukan pada tahap penulisan laporan ini. Oleh karena itu, bagian ini disiapkan sebagai template pengujian yang dapat langsung dilengkapi setelah eksperimen dijalankan. Pengujian dirancang untuk mencakup unit test backend, integrasi API, validasi frontend, dan perbandingan BFS/DFS.

4.5.1 Perintah Pengujian

Template Perintah Pengujian Backend

```
cd backend
go test ./src/parser -v
go test ./src/selector -v
go test ./src/traversal -v
go test ./... -v
```

Template Perintah Pengujian Frontend

```
cd frontend
bun run lint
bun run test
bun run build
```


4.5.2 Template Unit Test Backend

No.	Modul	Skenario	Ekspektasi	Status
1	Parser	Parsing HTML sederhana dengan tag bertingkat.	Root, child, depth, dan atribut terbentuk benar.	TBD
2	Parser	Parsing tag kosong seperti <code>img</code> dan <code>br</code> .	Tag kosong menjadi node tanpa mengganggu stack parent.	TBD
3	Selector	Tag, class, id, dan universal selector.	MatchSelector mengembalikan boolean sesuai selector.	TBD
4	Selector	Combinator <code>></code> , spasi, <code>+</code> , dan <code>~</code> .	Match mengenali relasi parent, ancestor, dan sibling.	TBD
5	Traversal	BFS tanpa limit.	Node dikunjungi per level dan semua match ditemukan.	TBD
6	Traversal	DFS tanpa limit.	Node dikunjungi mendalam dengan urutan sibling kiri-ke-kanan.	TBD
7	Traversal	Top- <i>N</i> dengan limit positif.	Traversal berhenti saat jumlah match mencapai limit.	TBD
8	Traversal	Selector kosong atau algoritma tidak valid.	Fungsi mengembalikan error yang sesuai.	TBD

4.5.3 Template Pengujian API

No.	Endpoint	Input	Ekspektasi	Status
1	GET <code>/api/health</code>	Tidak ada body.	Response status OK dan JSON status .	TBD
2	POST <code>/api/traverse</code>	URL valid, selector <code>a</code> , algoritma BFS.	Response berisi tree, log, metrik, algoritma, dan selector.	TBD
3	POST <code>/api/traverse</code>	Raw HTML, selector <code>.card</code> , algoritma DFS.	Response memiliki match sesuai class pada HTML.	TBD
4	POST <code>/api/traverse</code>	Input kosong.	Response error validasi.	TBD
5	POST <code>/api/traverse</code>	Selector kosong.	Response error validasi.	TBD
6	POST <code>/api/traverse</code>	Algoritma selain BFS/DFS.	Response error dari SearchDOM .	TBD

4.5.4 Template Pengujian Fungsional Frontend

No.	Fitur	Ekspektasi	Status
1	Health check backend	Status backend berubah menjadi online ketika <code>/api/health</code> dapat diakses.	TBD
2	Input URL	Form mengirim URL ke backend dan menampilkan hasil traversal.	TBD
3	Input raw HTML	Form mengirim string HTML sebagai mode <code>html</code> .	TBD
4	Input file HTML	Isi file dibaca dengan FileReader dan dikirim sebagai HTML.	TBD
5	Tab Graph	Pohon DOM tampil sebagai SVG dan node match ter-highlight.	TBD
6	Tab Tree	Struktur DOM dapat diekspansi dan node match terlihat.	TBD
7	Tab Log	Log traversal tampil sesuai urutan step dan dapat difilter match.	TBD
8	Error handling	Pesan error muncul saat backend offline atau input tidak valid.	TBD

4.5.5 Template Hasil Eksperimen BFS dan DFS

Tabel berikut dapat digunakan untuk mencatat perbandingan BFS dan DFS pada dokumen HTML yang sama. Kolom hasil diisi setelah pengujian dijalankan.

No.	Input	Selector	Algo	Visited	Match	Time
1	test/cases/case_1.html	article	BFS	TBD	TBD	TBD
2	test/cases/case_1.html	article	DFS	TBD	TBD	TBD
3	test/cases/case_2.html	card	BFS	TBD	TBD	TBD
4	test/cases/case_2.html	card	DFS	TBD	TBD	TBD
5	URL publik	a	BFS	TBD	TBD	TBD
6	URL publik	a	DFS	TBD	TBD	TBD

4.5.6 Placeholder Analisis Hasil

Setelah pengujian dilakukan, analisis dapat diisi dengan poin berikut:

- Apakah BFS dan DFS menghasilkan jumlah match yang sama untuk selector tanpa limit.
- Perbedaan urutan match antara BFS dan DFS berdasarkan traversal log.
- Pengaruh Top- N terhadap jumlah node yang dikunjungi.
- Perbedaan waktu eksekusi pada dokumen kecil dan dokumen besar.
- Kasus selector yang tidak didukung dan bagaimana aplikasi menampilkan hasilnya.
- Kesesuaian visualisasi frontend terhadap hasil traversal backend.

Bab 5

Kesimpulan dan Saran

5.1 Kesimpulan

Sebagai penutup, DOMTracer merupakan sebuah program yang mengemulasi cara kerja CSS Selector dalam memilih elemen-elemen bersesuaian dengan pada sebuah Document Object Model (DOM). Dalam program ini, DOM direpresentasikan sebagai struktur pohon dan kemudian diproses dengan menggunakan prinsip traversal graf/pohon berbasis algoritma Breadth-First Search (BFS) dan Depth-First Search (DFS). Ide besar ini kemudian diterjemahkan menjadi rancangan sistem yang modular melalui pemisahan komponen parser, selector, dan traversal agar alur pemrosesan HTML menjadi jelas dan terstruktur.

Pada sisi implementasi, kami berhasil membangun alur lengkap dari masukan HTML, proses parsing menjadi struktur node, pencarian elemen berdasarkan selector, hingga penyajian hasil melalui antarmuka CLI. Fitur-fitur pendukung seperti pilihan sumber input (URL, HTML manual, dan file), pembatasan jumlah hasil (Top-N), serta pencatatan traversal log menunjukkan bahwa solusi tidak hanya berjalan secara fungsional, tetapi juga memperhatikan aspek kegunaan dan observabilitas proses.

Selama pengerjaan, banyak ilmu yang diperoleh, terutama dalam menghubungkan konsep algoritma yang dipelajari di kelas dengan kebutuhan rekayasa perangkat lunak nyata. Kami belajar bahwa pemilihan strategi traversal memengaruhi urutan kunjungan dan performa praktis, bahwa desain struktur data sangat menentukan kemudahan pengembangan fitur lanjutan, serta bahwa pengujian berbasis kasus berperan penting untuk menjaga ketahanan program pada input yang beragam, termasuk kasus batas dan input tidak valid.

Refleksi utama dari tugas besar ini adalah pentingnya berpikir sistematis dari definisi masalah, perancangan solusi, implementasi bertahap, hingga validasi hasil. Pengalaman ini menjadi bekal berharga untuk proyek-proyek di masa depan, khususnya dalam menerapkan algoritma secara kontekstual, menulis kode yang mudah dirawat, dan membangun kebiasaan verifikasi yang disiplin sebelum perangkat lunak digunakan lebih luas.

5.2 Saran

Program ini masih dapat dikembangkan lebih lanjut agar semakin kuat dan mendekati kebutuhan aplikasi nyata. Beberapa pengembangan yang disarankan adalah sebagai berikut.

1. Menambah dukungan CSS selector yang lebih lengkap, seperti selector gabungan yang lebih kompleks, pseudo-class tertentu, dan validasi sintaks selector yang lebih informatif.
2. Mengoptimalkan performa parser dan traversal untuk dokumen HTML yang sangat besar, misalnya melalui pendekatan streaming parse, caching hasil seleksi tertentu, atau strategi pruning pada proses pencarian.
3. Memperluas pengujian otomatis, termasuk benchmark performa, property-based testing, dan pengujian terintegrasi pada pipeline CI.

Tugas ini disusun sepenuhnya tanpa bantuan kecerdasan buatan (Generative AI), melainkan hasil pemikiran dan analisis mandiri.

Bandung, 19 April 2026

Moh. Hafizh Irham Perdana

Aufa Rienaldifaza Ahmad

Muhammad Aufar Rizqi Kusuma

Bab 6

Lampiran

6.1 Tabel Checklist

No	Poin	Ya	Tidak
1	Aplikasi berhasil di kompilasi tanpa kesalahan	✓	
2	Aplikasi berhasil dijalankan	✓	
3	Aplikasi dapat menerima input URL web, pilihan algoritma, CSS selector, dan jumlah hasil	✓	
4	Aplikasi dapat melakukan scraping terhadap web pada input	✓	
5	Aplikasi dapat menampilkan visualisasi pohon DOM	✓	
6	Aplikasi dapat menelusuri pohon DOM dan menampilkan hasil penelusuran	✓	
7	Aplikasi dapat menandai jalur tempuh oleh algoritma	✓	
8	Aplikasi dapat menyimpan jalur yang ditempuh algoritma dalam traversal log	✓	
9	[Bonus] Membuat video		✓
10	[Bonus] Deploy aplikasi	✓	
11	[Bonus] Implementasi animasi pada penelusuran pohon	✓	
12	[Bonus] Implementasi multithreading		✓
13	[Bonus] Implementasi LCA Binary Lifting		✓
14	[Bonus] Docker	✓	

6.2 Tautan GitHub

Link GitHub

6.3 Web Deploy (via Azure)

domtracer.com

Bibliografi

- [1] WHATWG, “DOM Standard,” <https://dom.spec.whatwg.org/>, 2026, accessed: 24-Apr-2026.
- [2] —, “HTML Standard,” <https://html.spec.whatwg.org/>, 2026, accessed: 24-Apr-2026.
- [3] World Wide Web Consortium, “Selectors Level 4,” <https://www.w3.org/TR/selectors-4/>, 2022, accessed: 24-Apr-2026.
- [4] T. Cormen, C. Leiserson, R. Rivest, and C. Stein, *Introduction to Algorithms*, 3rd ed. Massachusetts Institute of Technology, 2009.